Cursor IDE

A Structured Study Packet · for the power user brushing up

Built with an 8-principle learning method · print / read edition

---

FRONT MATTER

How to use this packet

This is a learning system, not a feature list. Work it in two passes. **Step 1 (Principles 1–4)** rebuilds a correct mental model of how Cursor works — its primitives, context engine, and agent loop. **Step 2 (Principles 5–8)** makes it stick through retrieval, spacing, mixing, and overlearning. The retention is in Step 2.

Content verified mid-2026 (Cursor 3.x, Composer 2.5, Cloud Agents, `.cursor/rules/`, MCP/hooks/skills, Hobby/Pro/Business/Enterprise pricing). Cursor ships fast — treat exact numbers as "true as of now," concepts as durable.

The 8 principles at a glance

Understanding — 1) Map of the system · 2) Clear explanations · 3) Different media · 4) Short lessons

Automaticity — 5) Test yourself · 6) Wait to review · 7) Mix it up · 8) Don't stop

The four lenses used throughout

Solo senior dev — flow & craft | Manager — team standards & review | CEO — cost, security, ROI | Unlimited — extend & orchestrate

STEP 1 — UNDERSTANDING

Goal: a correct, simple mental model — what Cursor is made of and how a request flows through it.

PRINCIPLE 1

A map of the system

Cursor is an **AI-native code editor** — a fork of VS Code by Anysphere that keeps VS Code's extensions, themes, and keybindings, but rebuilds the editing loop around AI. Three layers: *interaction primitives* (how you talk to it), the *context engine* (what it knows about your code), and the *agent runtime* (what it can do on its own).

How a request flows

#	Stage	What happens
1	Intent	You trigger a primitive: Tab, Inline Edit (`Ctrl/Cmd+K`), Chat (`Ctrl/Cmd+L`), or Agent/Composer.
2	Context assembly	Cursor gathers open files, `@`-mentions, the semantic codebase index, and matching rules.
3	Rules + routing	Active rules are prepended; the request routes to a model (Composer, Claude, GPT-5, o-series), optionally Max Mode.
4	Generation	The model proposes edits, an answer, or a multi-step plan.
5	Action (agent only)	Agent runs terminal commands, reads output, edits files, browses `@web`, calls MCP tools — looping until done.
6	Review & apply	You see a diff and accept/reject per hunk.

The components

Component	What it is
Tab	Predictive autocomplete suggesting multi-line / multi-location edits, not just the next token.
Inline Edit (`Ctrl/Cmd+K`)	Natural-language edit on a selection or at the cursor, in place.
Chat / Ask (`Ctrl/Cmd+L`)	A read-oriented side conversation about the codebase; doesn't act on its own.
Composer / Agent	Agentic mode: plans and executes multi-file changes with tools (terminal, files, web, MCP).
Context engine	`@`-symbols + the embeddings-based codebase index for semantic retrieval.

Rules	`.cursor/rules/` (and legacy `.cursorrules`) — persistent instructions injected into context.
Extensions (MCP, hooks, skills)	Plug external tools, lifecycle automation, and reusable capabilities into the agent.
Cloud / Background Agents	Agents in isolated cloud VMs, parallel across repos, reporting back asynchronously.

> **Map takeaway**
>
> Everything in Cursor is one of three things: a **primitive** you trigger, the **context** it assembles, or the **agent loop** it runs.

PRINCIPLE 2

Clear, simple explanations

What makes Cursor different from "VS Code + Copilot"?

Copilot bolts AI onto an editor as a suggestion engine. Cursor rebuilds the editor *around* an agent: a unified context engine indexes your whole repo, and an agent runtime runs commands and edits many files in a loop. The difference is architectural — Cursor treats the editor as a runtime the agent operates, not a textbox it autocompletes.

What is Tab, really?

Tab is Cursor's predictive editing model. Beyond completing the current line, it predicts the *next edit* — often multi-line or jumping to another location implied by your change (e.g. updating a call site after a rename). It's the most-used feature; recent Tab reached very high acceptance (~72%), which is why it feels like the editor reads your intent.

Inline Edit vs Chat vs Agent — when does each fire?

Inline Edit (`Ctrl/Cmd+K`) = a surgical in-place change. **Chat/Ask** (`Ctrl/Cmd+L`) = questions and exploration; reads and explains but doesn't act. **Agent/Composer** = "go do it" multi-file work where the model plans, runs tools, and applies changes. Rule of thumb: *Ask to understand, Inline to tweak, Agent to build.*

How does Cursor know about my codebase?

It builds a **semantic index**: files are chunked and embedded into vectors so the model retrieves code by meaning, not filename. You steer it with `@`**-symbols** — `@file`, `@folder`, `@code`, `@docs`, `@git`, `@web`. Context hygiene is the biggest lever on output quality.

What is the context window, and what is Max Mode?

The **context window** is how much text the model holds at once — by default ~**200k tokens** (~15k lines). **Max Mode** expands that ceiling for large models at higher token cost. Use it for sprawling refactors; skip it for small edits where extra context just adds noise and spend.

What are Rules and why do they matter more than prompts?

Rules are persistent instructions injected into context on relevant requests. Modern Cursor uses `.cursor/rules/` — multiple `.mdc` files loading conditionally: *Always* (every request), *Auto-Attached* (when files matching a glob are in context), *Agent-Requested* (the model pulls it), or *Manual* (you `@`-mention it). Legacy single `.cursorrules` still works, but folder rules keep the system prompt small. Rules beat one-off prompts because they're versioned, shared, and automatic.

What is MCP, and how do hooks and skills extend it?

MCP (Model Context Protocol) is an open standard connecting the agent to external tools and data (databases, issue trackers, browsers, internal APIs) through a uniform interface. **Hooks** run your own logic at lifecycle points (before/after an edit or command) to gate or shape behaviour. **Skills** are reusable, pinnable capability bundles. Together they turn Cursor into a programmable agent platform — available on Pro and above.

What are Cloud / Background Agents?

A **Cloud Agent** runs in an isolated cloud VM with its own terminal, browser, and file system. It works across multiple repos in parallel and reports results back asynchronously — fire a task and keep working. "Build in Parallel" spawns subagents to tackle pieces concurrently.

How does Cursor cost money — requests, credits, seats?

Tiers: **Hobby** (free; limited Tab + slow requests), **Pro** (~$20/mo; unlimited Tab + a pool of fast requests, then a slow queue), **Business** (~$40/seat/mo; adds SSO, admin controls, zero-data-retention), **Enterprise** (custom). "Fast" requests are prioritised compute; heavy operations (Max Mode, big agent runs) consume more. *Tab is cheap and flat; agentic runs and Max Mode are where spend concentrates.*

Explanation takeaway

The one organising idea: **context quality drives everything. Common misconception:** that a bigger model or Max Mode fixes bad output — usually the fix is better-scoped context and a good rule, not more tokens.

PRINCIPLE 3

Use different media

One-line summary

Cursor is a VS Code–based editor rebuilt around an AI agent: a semantic index and `@`-context feed models that autocomplete (Tab), edit in place, and — in Agent mode — run tools to make multi-file changes, steered by persistent rules and extended by MCP, hooks, skills, and cloud agents.

Diagram — the request loop

```
 YOU
  |  trigger
  v
PRIMITIVE -- Tab . Inline(Ctrl+K) . Chat(Ctrl+L) . Agent
  |
  v
CONTEXT ENGINE
  |- @-symbols (file/folder/docs/web/git)
  |- semantic codebase index  (embeddings)
  '- RULES  (.cursor/rules/ -> Always | Auto | Requested | Manual)
  |
  v
MODEL ROUTING -- Composer . Claude . GPT-5 . o-series   [+ Max Mode = bigger window]
  |
  v
AGENT LOOP (agent only)
  run cmd -> read output -> edit files -> @web -> MCP tool --+
       ^_________________ repeat until done _________________|
  |
  v
DIFF -> you accept/reject per hunk
       (or dispatch a CLOUD AGENT: isolated VM, parallel, async)
```

Analogy

> A **workshop with a skilled apprentice**. *Tab* finishes your sentence as you work. *Inline Edit* hands them one part: "make this fit." *Chat* asks about the project without letting them touch anything. *Agent* says "build this shelf" and lets them fetch tools, cut wood, and check their work — while you inspect the result. *Rules* are the shop's posted standards. *MCP* gives them keys to the supply room. *Cloud agents* are a second apprentice working in another room, reporting back.

Comparison table — Cursor vs neighbours

Tool	Core model	Best at	Watch-out
Cursor	AI-native editor (VS Code fork) + agent runtime	Whole-repo agentic edits with tight review	Cost concentrates in agent/Max runs
GitHub Copilot	Autocomplete + chat on your editor	Inline suggestions, broad IDE support	Less of a unified repo-wide agent loop
VS Code (plain)	Editor; AI via extensions	Familiarity, ecosystem	No native agent/context engine
CLI agents (e.g. Claude Code)	Terminal-driven agent	Scripting, headless/CI flows	No rich in-editor diff/Tab UX

> **Media takeaway**
> Remember the loop: **primitive → context → model → (agent loop) → diff.**

PRINCIPLE 4

Short lessons

Lesson 1 — The four primitives, and choosing between them

Tab predicts edits as you type; Inline Edit (Ctrl+K) changes a selection in place; Chat (Ctrl+L) explains without acting; Agent/Composer plans and executes multi-file work. Pick by *blast radius*: bigger and more autonomous → move toward Agent.

Lesson 2 — Context is something you steer

The index gives the model a searchable memory of your repo, but best results come from pointing explicitly with `@file`, `@folder`, `@docs`, `@git`, `@web`. Less but relevant beats dumping everything.

Solo senior dev

Lesson 3 — Rules are your standards-as-code

Put conventions in `.cursor/rules/` as scoped `.mdc` files: *Always* for universal style, *Auto-Attached* globs for per-area rules, *Agent-Requested* for situational guidance. This is how a team gets consistent agent output without everyone re-prompting. Manager

Lesson 4 — Models and Max Mode are a cost/benefit dial

Cursor routes among Composer, Claude, GPT-5, and o-series; Max Mode trades money for a much larger context window. Reach for Max on large cross-cutting refactors; otherwise the default window is faster, cheaper, often cleaner. CEO

Lesson 5 — Extend the agent with MCP, hooks, and skills

MCP connects external tools/data behind one protocol; hooks run your logic at lifecycle points (gate a command, lint after an edit); skills bundle reusable capabilities you pin. This makes Cursor a platform tailored to your stack. Unlimited

Lesson 6 — Cloud agents change the unit of work

Background/Cloud Agents run in isolated VMs, in parallel, across repos, reporting asynchronously. The skill shifts from "writing code with help" to "*dispatching and reviewing* agent work" — decomposition and review become the bottleneck. Unlimited Manager

> **Short-lessons takeaway**
>
> Choose primitives by blast radius, steer context deliberately, encode standards as rules, treat Max Mode as a cost dial, extend via MCP/hooks/skills, and learn to dispatch and review cloud agents.

STEP 2 — AUTOMATICITY

Understanding fades. Retrieval, spacing, mixing, and overlearning turn it into durable recall. This is the half most people skip — and the half that works.

PRINCIPLE 5

Test yourself

Cover the answers, recall from memory, then check. The struggle is what builds the memory.

Q1. Cursor is a fork of which editor, and who makes it?

A. A fork of **VS Code**, made by **Anysphere**. Keeps VS Code's extensions, themes, keybindings.

Q2. Name the four interaction primitives and two keyboard triggers.

A. **Tab**, **Inline Edit** (Ctrl/Cmd+K), **Chat/Ask** (Ctrl/Cmd+L), **Agent/Composer**.

Q3. What does Tab predict beyond the current line?

A. The **next edit** — multi-line, and edits at another location implied by your change (e.g. a call site after a rename).

Q4. How does Cursor retrieve relevant code semantically?

A. A **semantic index**: files chunked and embedded as vectors, retrieved by meaning. Steered with `@`-symbols.

Q5. Default context window, and what does Max Mode do?

A. ~**200k tokens** (~15k lines). **Max Mode** expands the ceiling for large models at higher cost — for sprawling refactors.

Q6. Name the four rule activation types in `.cursor/rules/`.

A. **Always**, **Auto-Attached** (glob), **Agent-Requested**, **Manual** (@-mention).

Q7. What does MCP stand for and do?

A. **Model Context Protocol** — an open standard connecting the agent to external tools/data through a uniform interface.

Q8. In Agent mode, list three tools the agent can use.

A. Any three: run **terminal commands** + read output, **file ops**, `@web`, the **semantic index**, **MCP tools**.

Q9. What distinguishes a Cloud/Background Agent from the in-editor agent?

A. Runs in an **isolated cloud VM**, works across **multiple repos in parallel**, reports back **asynchronously**.

Q10. Which paid tier first adds SSO + zero-data-retention, and roughly the cost?

A. **Business**, ~**$40/seat/mo** — adds SSO, admin controls, ZDR over Pro.

Q11. Why do rules beat repeating instructions each prompt?

A. Rules are **persistent, versioned, shared, automatic** — injected on relevant requests without re-typing.

Q12. State the single biggest lever on Cursor output quality.

A. **Context quality** — scoping the right code/conventions in, not picking a bigger model or Max Mode.

Flashcards

Cover the right column, recall, then check.

Prompt	Answer
What is Cursor, in one line?	An AI-native code editor — a VS Code fork by Anysphere rebuilt around an agent + context engine.
Tab vs Inline Edit?	Tab predicts your next edit as you type; Inline Edit (Ctrl+K) applies a natural-language change in place.
Chat/Ask vs Agent?	Ask reads & explains without acting; Agent plans & executes multi-file changes with tools.
What feeds the model context?	@-symbols + the semantic codebase index + active rules.
@-symbols, name four	@file, @folder, @docs, @git, @web (any four).
Default context window?	~200k tokens (~15k lines).
What does Max Mode do?	Expands the context window for large models, at higher token cost — for big refactors.

Where do rules live now?	.cursor/rules/ as scoped .mdc files (legacy: single .cursorrules).
Four rule activation types	Always, Auto-Attached (glob), Agent-Requested, Manual.
What is MCP?	Model Context Protocol — open standard connecting the agent to external tools/data.
Hooks vs Skills?	Hooks run your logic at agent lifecycle points; Skills are reusable, pinnable capability bundles.
Agent-mode tools (3)	Run terminal cmds + read output, file ops, @web, semantic index, MCP tools.
What is a Cloud Agent?	An agent in an isolated cloud VM — parallel, multi-repo, asynchronous reporting.
Tiers & ZDR	Hobby(free) · Pro(~$20) · Business(~$40/seat, +SSO +ZDR) · Enterprise.
Biggest lever on output quality?	Context quality — scope the right code/rules in; not a bigger model or Max Mode.

PRINCIPLE 6

Wait to review

Space reviews at expanding gaps. Tick each session.

When	What to do	Done
Today	Read the whole packet. Do the Principle 5 quiz once, open-book on misses.	☐
Day 1	Flashcards cold (cover answers). Re-draw the request-loop diagram from memory.	☐
Day 3	Quiz closed-book. Re-read only the ideas you missed in Principle 2.	☐
Day 7	Interleaved quiz (Principle 7). Note any lens you're weak on.	☐
Day 14	Flashcards + teach one concept aloud (MCP, or the Max-Mode trade-off).	☐
Day 30	Cold full self-test: both quizzes, no notes. Anything shaky → back into rotation.	☐

Why spacing works: each near-forget-then-retrieve signals the memory is worth keeping. Re-reading feels easier but builds far less durable recall.

PRINCIPLE 7

Mix it up

This quiz jumps between primitives, context, rules, cost, security, and extension — out of order — so you practise *choosing* the right idea.

B1. CEO weighing rollout: which tier gives ZDR + SSO, and why care?

A. **Business** (~$40/seat). ZDR + SSO matter for **compliance and IP protection** when code can't leave under retention.

B2. A teammate's agent keeps ignoring your naming convention. Fix at team level — how?

A. Add an **Always** (or glob Auto-Attached) rule in `.cursor/rules/`, commit it. Every agent run inherits it.

B3. Rename a variable in a 12-line function. Which primitive, and why not Agent?

A. **Inline Edit (Ctrl+K)** or **Tab** — minimal blast radius. Agent is overkill and adds review cost.

B4. Agent gives wrong output on a big refactor. Two fixes *before* Max Mode?

A. Tighten **context scope** (point at the right files via `@`) and add/clarify a **rule**. Context > raw tokens.

B5. Architectural difference between Cursor and "VS Code + Copilot"?

A. Cursor is built **around an agent** (unified context engine + agent runtime running tools); Copilot bolts suggestions onto the editor.

B6. Agent must query internal Postgres and Jira. What connects those?

A. **MCP** servers expose those systems to the agent through one protocol.

B7. Why dispatch a Cloud Agent instead of the in-editor agent?

A. For **long/parallel work across repos asynchronously** in isolated VMs while you keep working; review later.

B8. Distinguish Chat/Ask from Agent in one sentence.

A. Ask **reads and explains** without acting; Agent **plans and executes** changes with tools.

B9. Cost intuition: where does spend concentrate?

A. Tab is cheap/flat; **agentic runs and Max Mode** consume most (fast requests / larger context).

B10. A rule that only applies when editing test files — which type and mechanism?

A. **Auto-Attached** with a **glob** (e.g. `*.test.ts`) in a `.cursor/rules/` `.mdc` file.

B11. A hook use-case that improves safety on an autonomous agent.

A. Gate/validate a terminal command before it runs, or auto-lint/format after an edit — logic at lifecycle points.

B12. Solo senior dev: what daily habit most improves results?

A. **Context hygiene** — explicitly `@`-mention relevant files/docs and keep rules current, not dump the whole repo.

Why mixing helps: real work never arrives labelled by topic. Interleaving trains the cue "which idea fits *this* situation" — the skill you actually use.

PRINCIPLE 8

Don't stop

Most people quit at first correct answer. Practice *after* first success is what makes recall fast and automatic under pressure.

Stage	You can…	Do this
First correct	Recall with effort and hints	Repeat it later the same day; don't assume it's learned.
Comfortable	Answer cold, slowly	Speed up: flashcards until a clean round; explain the loop diagram unaided.

Automatic	Answer instantly and teach it	Maintain with monthly cold tests; apply in real Cursor work.

Overlearning plan

- Run the flashcards until **three flawless rounds** in a row.
- Once a month, a **cold self-test** of both quizzes — no notes.
- **Teach it:** explain MCP, the rule activation types, and the Max-Mode trade-off to a colleague.
- **Apply it:** on your next real task, pick the primitive by blast radius and write one new `.cursor/rules/` file.

> **Final takeaway**
>
> Step 1 gave the model — primitive → context → model → agent loop → diff, extended by rules, MCP, and cloud agents. Step 2 makes it permanent: **test, space, mix, overlearn.** Knowledge becomes durable through practice *after* first success — don't stop at "I understood it."

APPENDIX

Quick-reference glossary

Term	Plain-language definition
Anysphere	The company that builds Cursor.
Tab	Predictive editing suggesting your next edit, often multi-line or at another location.
Inline Edit (Ctrl+K)	Natural-language edit applied in place to a selection or at the cursor.
Chat / Ask (Ctrl+L)	A read-oriented conversation about the codebase; explains, doesn't act.
Composer / Agent	Agentic mode that plans and executes multi-file changes using tools.
Codebase index	Embeddings-based semantic memory of your repo for meaning-based retrieval.
@-symbols	Mentions (@file, @folder, @docs, @git, @web) that inject specific context.
Context window	Max text a model holds at once; ~200k tokens by default.

Max Mode	Expands the context window for large models, at higher token cost.
Rules	Persistent instructions in .cursor/rules/ (or legacy .cursorrules) injected into context.
Activation types	How a rule loads: Always, Auto-Attached (glob), Agent-Requested, Manual.
MCP	Model Context Protocol — open standard to connect the agent to external tools/data.
Hooks	Your logic run at agent lifecycle points to gate or shape behaviour.
Skills	Reusable, pinnable capability bundles for the agent.
Cloud / Background Agent	Agent in an isolated cloud VM; parallel, multi-repo, asynchronous.
Fast vs slow requests	Prioritised vs queued AI compute; Pro includes a fast pool then a slow queue.
Zero-data-retention (ZDR)	Mode (Business+) where your code isn't retained — for compliance/IP.

Further study: read Cursor's official docs on Rules and MCP, then build one real `.cursor/rules/` file and connect one MCP server.
